

COMPILER DESIGN

[CS-603]

UNIT - 1

INTRODUCTION TO COMPILING & LEXICAL ANALYSIS

SYLLABUS -

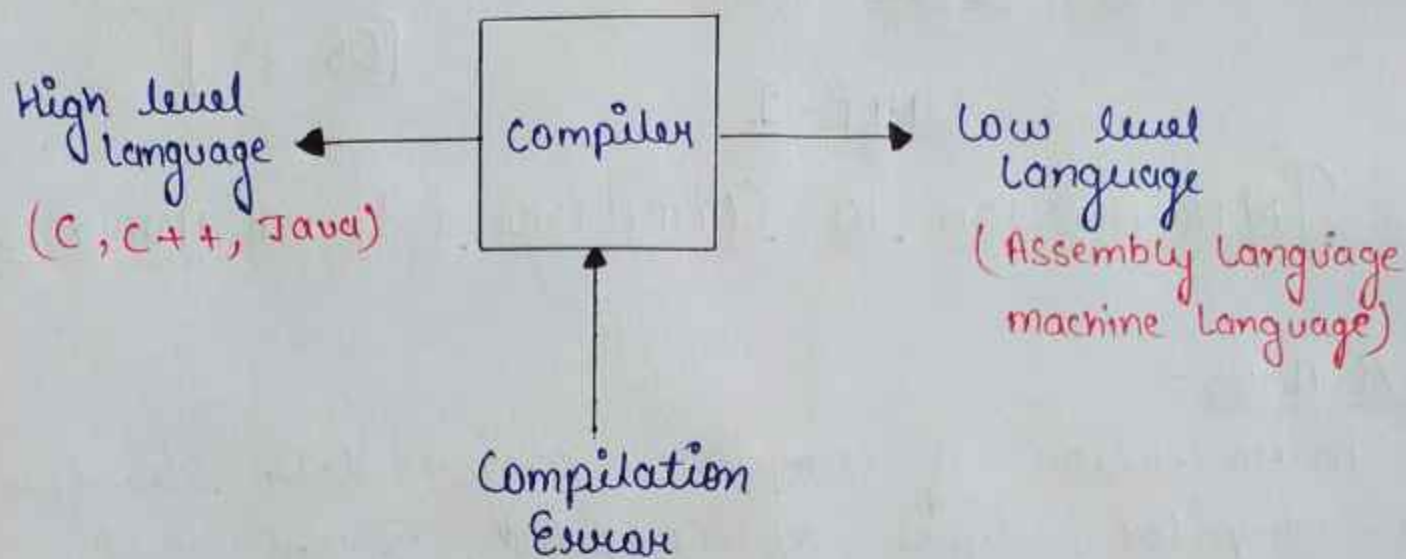
Introduction of compiler, major data structure in compiler, types of compiler, Front-end and Back-end of compiler, Compiler structure: analysis-synthesis model of compilation, various phases of a compiler
Lexical analysis: Input buffering, specification & Recognition of Tokens, Design of a Lexical Analyzer generator, LEX.

INTRODUCTION OF COMPILER

Compiler is a software which converts a program written in high level language (source language) to low level language (Object / Target / machine language).

So that the computer can understand and execute it.

In simple words, a compiler translates the entire source program into machine code before execution.



Computers only understand binary language (0 and 1), so a compiler acts as a translator between human-readable code and machine code.

Example:-

A program written in C (programming language) is converted into machine code using a compiler such as GCC (GNU Compiler Collection).

Basic working of a Compiler -

The compiler reads the source code, checks it for errors & then converts it into the machine code.

Steps -

1. The programmer writes the program in a high-level language.
2. The compiler checks the program for syntax errors.
3. If no errors are found, the compiler converts the program into object code or machine code.
4. The computer executes the machine code to produce the output.

Language Processing System

A Language Processing system is a collection of programs used to translate a source program written in a high-level language into machine language so that the computer can execute it.

It includes different tools such as:-

- Preprocessor
- Compiler
- Assembler
- Linker
- Loader

These tools are work together to convert a program into an executable program.

Components of Language Processing system -

1. Preprocessor

The Preprocessor processes the source code before compilation.

It performs tasks such as:

- File inclusion
- Macro expansion
- Removing comments
- Conditional compilation

Example in C -

```
#include <stdio.h>
```

2. Compiler

The compiler converts the high-level language program into assembly language or machine code.

It also checks for:

- Syntax error
- Semantic error

Example compiler:-

GCC (GNU compiler Collection)

3. Assembler

The assembler converts assembly language into machine language.

Assembly language uses symbolic instructions like:

mov A, B

ADD A, C

The assembler converts these instructions into binary machine code.

4. Linker

The linker combines different object files and libraries into a single executable program.

for example:-

- Program file
- Library functions
- Header files.

All these are linked together by the linker.

5. Loader

The loader loads the executable program into main memory (RAM) so that the CPU can execute it.

Once the program is loaded into memory, it starts execution & produces the output.

MAJOR DATA STRUCTURES USED IN COMPILER

A compiler uses several data structures to store & manage information during the compilation process.

These data structures help the compiler analyze the program, store identifiers, represent program structure & generate machine code efficiently.

1. Symbol Table -

Stores information about variables, functions and identifiers.

Example stored information:

- Variable name
- Data type
- Memory location
- Scope

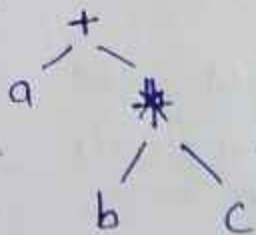
2. Syntax Tree -

A tree structure used to represent the hierarchical structure of the program.

Example for Expression:-

$$a + b * c$$

Syntax Tree -



3. Stack -

used in parsing & expression evaluation.

4. Hash Table -

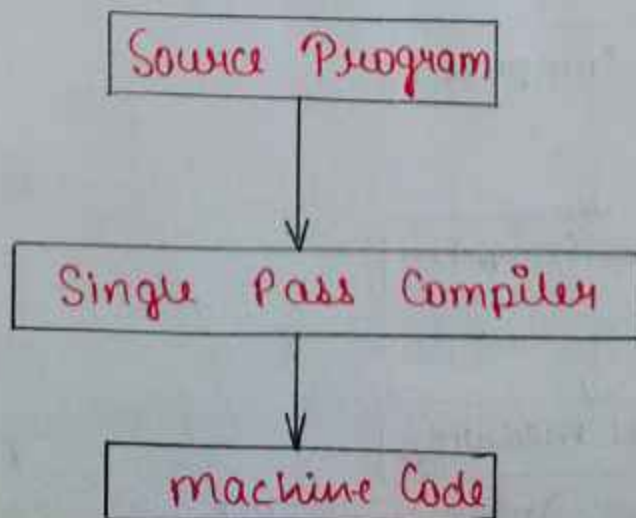
used for fast searching in symbol tables.

TYPES OF COMPILER

Different compilers are designed for different purposes, depending on how & where the program will run.

1. Single Pass Compiler

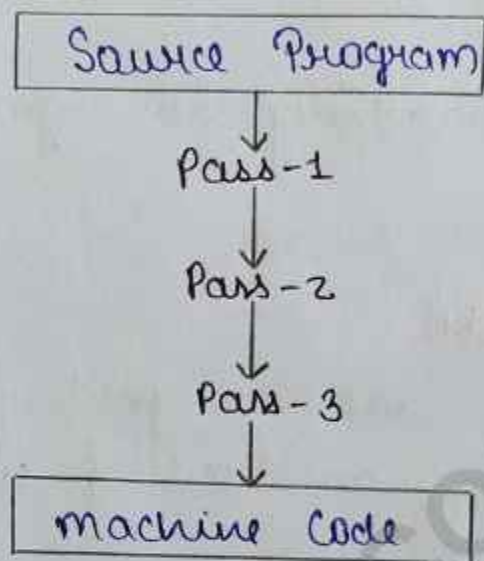
A Single Pass compiler reads the source program only once and converts it directly into machine code.



2. Multi Pass Compiler

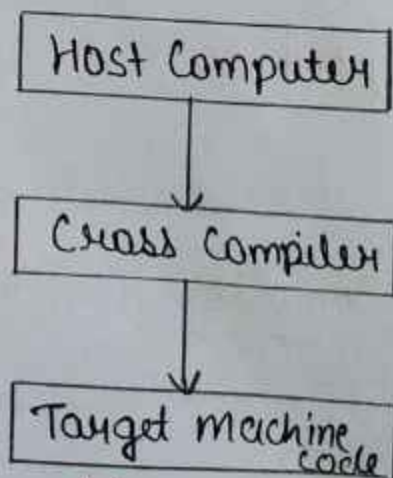
A multi pass compiler processes the source program multiple times to generate machine code.

Each pass performs a different task such as analysis, optimization or code generation.



3. Cross Compiler

A cross compiler is a compiler that runs on one machine but produces machine code for another machine.



(for another system)

4. Bootstrap Compiler

A Bootstrap compiler is a compiler that is written in the same language that it compiles.

This process is called Bootstrapping.

ANALYSIS AND SYNTHESIS MODEL OF COMPILATION

The Analysis & Synthesis model of Compilation explains how a compiler processes a program in two main phases:

1. Analysis Phase (Front-End)

2. Synthesis Phase (Back-End)

In this model, the compiler first analyzes the source program to understand its structure and then generates the target machine code.

1. Analysis Phase (Front End)

The Analysis Phase is the first part of the compilation process.

It reads the source program, checks for errors and converts it into an intermediate representation.

This phase is also called the Front End of the compiler.

Functions-

- Breaks the source code into tokens
- Checks syntax & semantics errors.
- Build symbol table
- Generates intermediate code

Components of Analysis Phase-

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis.

2. Synthesis Phase (Back End)

The Synthesis Phase generates the target machine code from the intermediate representation.

This Phase is also called the Back End of the compiler.

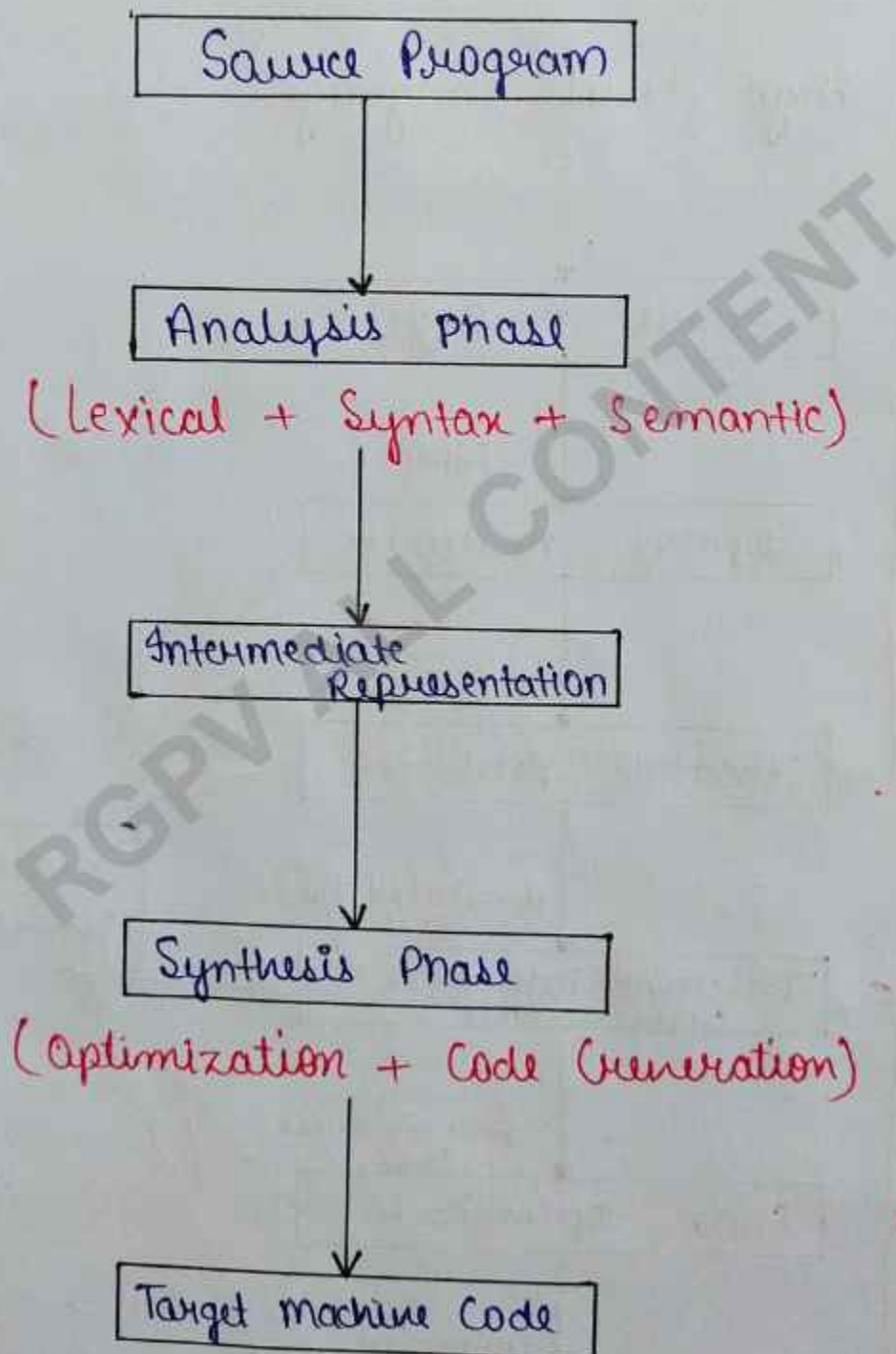
Functions-

- Optimizes intermediate code
- Generates machine code
- Produces final executable program.

Components of Synthesis Phase -

1. Code optimization
2. Code Generation

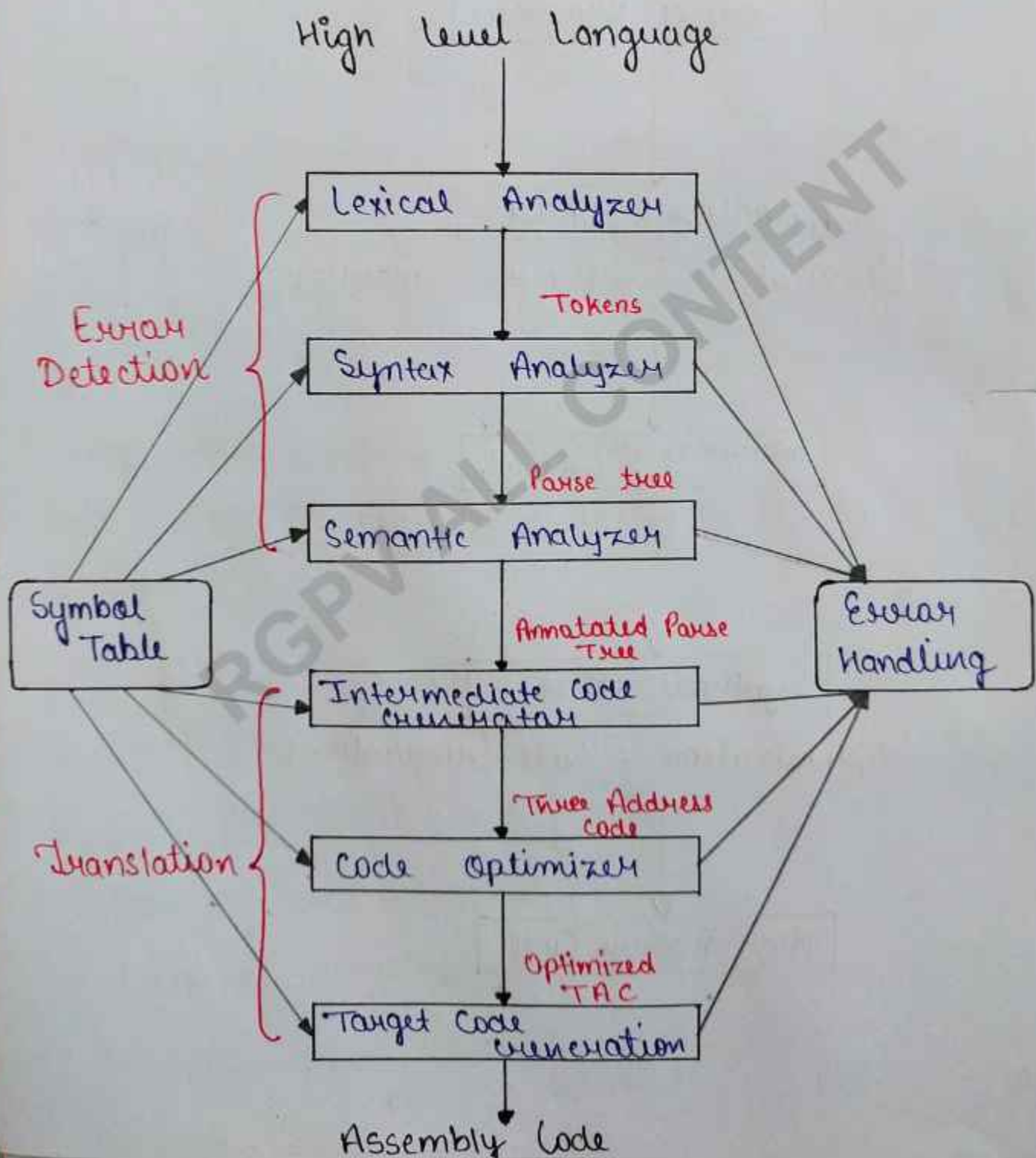
Diagram of Analysis - synthesis model -



PHASES OF COMPILER (Most Imp)

The translation of high level language to low level language divided into small-small modules.

Each module is called as one phase of the compiler.

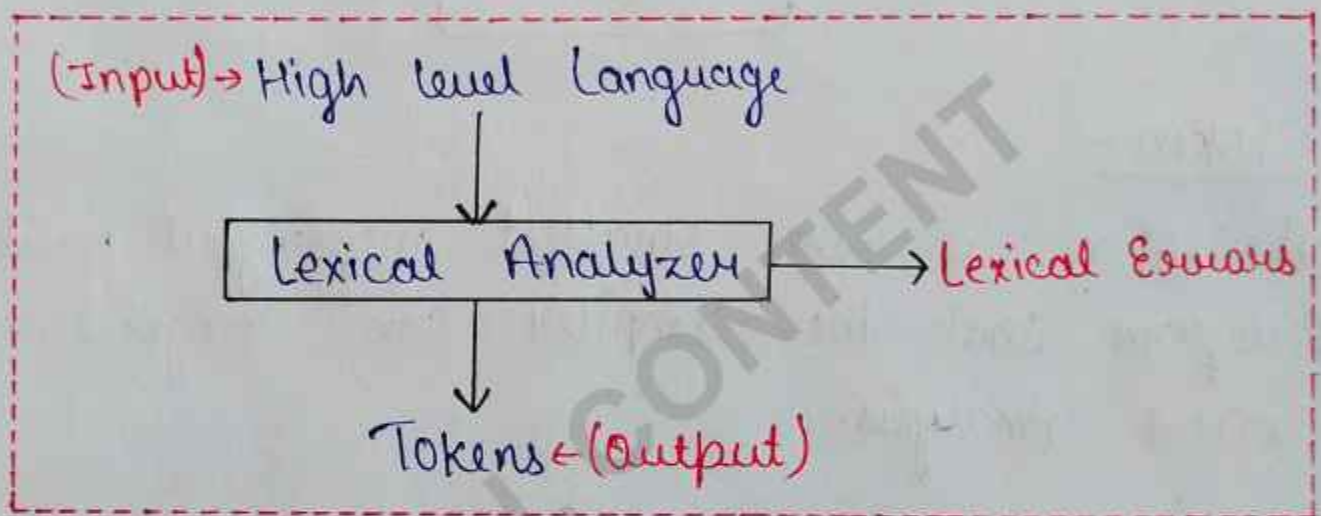


1. Lexical Analyzer

Lexical Analyzer is the first phase of the compiler.

It takes High level Language as input and generated tokens in the form of output

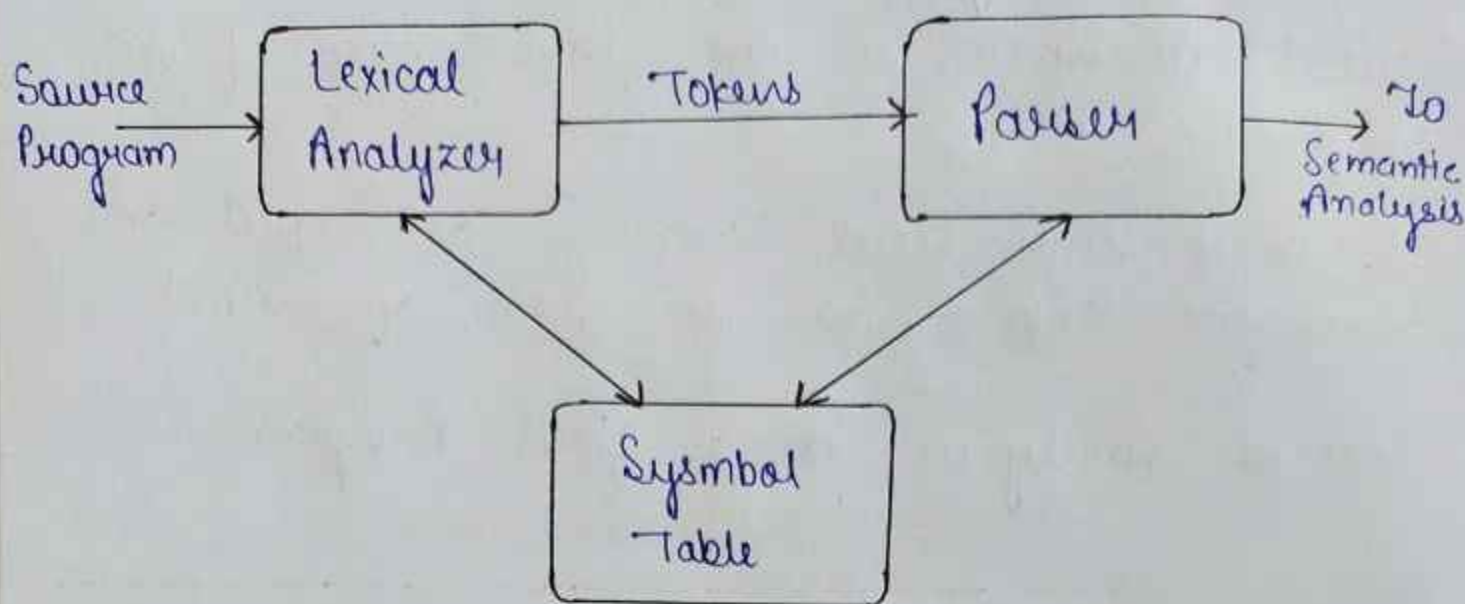
Lexical Analyzer is a first program.



Functionality of Lexical Analysis (or) Scanner

- 1) It produces tokens as output
- 2) It detects lexical errors.
- 3) It eliminates comments lines
- 4) It eliminates white space characters
- 5) It construct symbol table.

Role of Lexical Analyzer -



Token -

A Token is the smallest meaningful unit of a program that the compiler can recognize during lexical analysis.

When the lexical analyzer reads the source code, it breaks the program into tokens & sends them to the parser.

Typical Tokens are:

- 1) Identifiers (xyz, ab)
- 2) Keywords (int, float, for) / (reserved keywords)
- 3) Operators (+, *, =, ÷)
- 4) Constant (10, 20, 30) / String constant
- 5) Special Symbol ((,) { } ;)

1) Operator -

Increment / Decrement

++ --

Arithmetic operator

+ - * / %

Relational operator

< <= > >= == !=

Assignment operator

= += -= *= /= %=

Bitwise operator

<< >>

Example:-

a + b → 3 tokens

x = y + + + + + z ; → 8 tokens.

2) Identifiers -

- 1) Identifier can be formed using letter & digit.
- 2) Identifier can start with letter only (not digit)
- 3) Keyword name is not allowed as identifier
- 4) Identifier can't include any special characters

Example:-

int xyzabc xa ;
 id id

No. of tokens ⇒ 4.

3) Keywords -

Keywords are reserved words of a programming language that have predefined meanings.

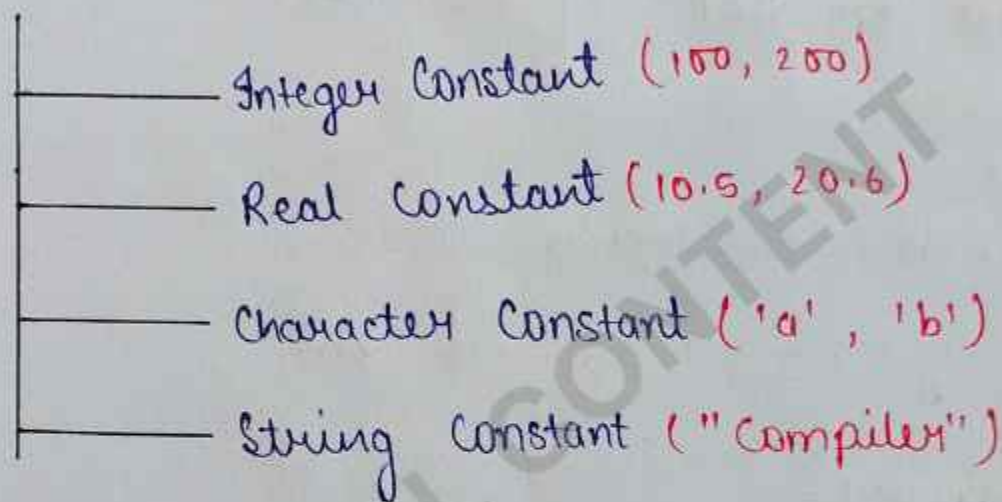
Example:-

int
float

if
while

return
for

4) Constant -



Example:-

int a = "Compiler"; → 5 tokens

5) Special Symbol -

) → consider as 1 token

(

{

}

;

,

Structure of a Token-

A token usually has two parts:

<Token name, Attribute Value>

Example of Tokens-

int d = "hello"; $\rightarrow$ 5 tokens

int a, b; $\rightarrow$ 5 tokens

a = b; + c; $\rightarrow$ 7 tokens

Consider the C Program-

<u>main()</u>	1 $\rightarrow$ 3 tokens
{	2 $\rightarrow$ 1 token
<u>int</u> <u>x</u> <u>=</u> <u>10</u> ;	3 $\rightarrow$ 5 tokens
<u>x</u> <u>=</u> <u>x</u> <u>+</u> <u>y</u> <u>+</u> <u>z</u> ;	4 $\rightarrow$ 8 tokens
}	5 $\rightarrow$ 1 token

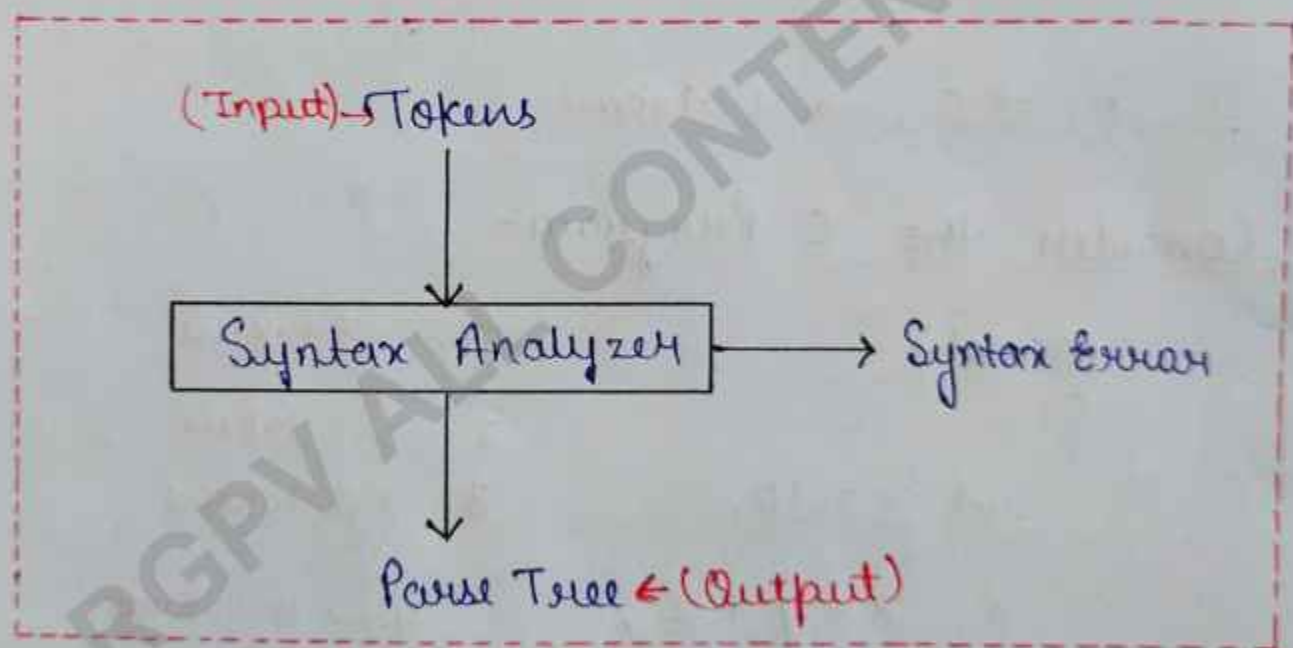
Total No. of tokens = 18 tokens.

2. Syntax Analyzer

The Syntax Analyzer is the second phase of the compiler.

It takes tokens produced by the lexical analyzer and checks whether they follow the grammar rules of the programming language.

This process is called parsing and the program that performs it is called a parser.



Purpose of Syntax Analyzer -

The Syntax Analyzer performs the following tasks -

- Checks correct grammatical structure of the program.
- Detects syntax errors.

- Creates a Parse tree or Syntax tree.
- Passes the structure to the semantic analyzer.

Example of Syntax Checking -

Program statement:

$a = b + c ;$

This statement follows the correct syntax of (Programming language)

So, it is accepted by the syntax analyzer.

Incorrect statement:

$= ab + c ;$

This statement violates grammar rules and will produce a syntax error.

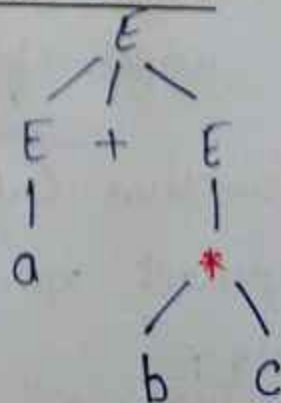
Parse Tree -

The Parse tree is a hierarchical tree that represents the syntactic structure of a program according to grammar rules.

Example Expression:-

$a + b * c$

Parse tree:-



where E represents an expression

3. Semantic Analyzer

The Semantic Analyzer is the third phase of the compiler.

It checks the meaning of the program statements after syntax analysis.

While the syntax analyzer checks grammar rules, the semantic analyzer verifies the statements are logically correct & meaningful.

Functions of Semantic Analyzer-

The Semantic ~~error~~ Analyzer performs the following task-

1. Type Checking

Ensures that operations are performed on compatible data types.

2. Checking variable declaration

Ensures variables are declared before use.

3. Scope Resolution

Checks the scope of variables (local or global).

4. Checking Function Call

Ensures correct number & types of arguments.

Example of Semantic Error -

```
int a;  
a = "hello";
```

A string value cannot be assigned to an integer variable.

This error is detected by the semantic analyzer.

```
int a;  
a = 10;
```

Here the assignment is semantically correct because the data type match.

4. Intermediate Code Generator

The Intermediate Code Generator is a phase of the compiler that converts the source program into an intermediate representation (IR).

The Intermediate code is machine-independent, meaning it does not depend on any specific computer architecture.

The intermediate code is later used by the code optimization & code generation phases to produce machine code.

Purpose of Intermediate Code-

The Intermediate code is used because:

- It makes compiler design easier.
- It allows machine - independent optimization.
- It simplifies translation to different machine codes.

Example -

Expression -

$$a + b * c$$

Intermediate Code (Three Address Code) -

$$t_1 = b * c$$

$$t_2 = a + t_1$$

Here,

t_1 and t_2 are temporary variables used for to store intermediate results.

5. Code Optimizer

Code Optimization is the phase of the compiler that takes intermediate code so that the final program runs faster and uses fewer resources such as memory and CPU time.

The code optimizer modifies the intermediate code without changing the output of the program.

Purpose of Code Optimizer -

The main goals of code optimization are -

- Reduce execution time
- Reduce memory usage
- Improve program efficiency.
- Remove unnecessary operations.

Example of Code Optimization -

Before Optimization:

$$t1 = a * 1$$

$$t2 = b + 0$$

$$t3 = t1 + t2$$

After Optimization:

$$t1 = a$$

$$t2 = b$$

$$t3 = a + b$$

Here unnecessary operations are removed, making the program more efficient.

6. Target Code Generation

Target Code Generation is the final phase of the compiler.

In this phase, the optimized intermediate code is converted into machine code or assembly language that can be executed by the computer.

The output produced is called the target program.

Functions of Code Generator-

The code generator performs the following tasks:

- Converts intermediate code into machine instructions.
- Allocates memory locations for variables.
- Selects efficient machine instructions.
- manages CPU registers.
- Produces final executable program

Example:-

Intermediate Code:

$t1 = b * c$

$t2 = a + t1$

Target machine Code:

LOAD b

MUL c

ADD a

STORE t2

Here the compiler converts intermediate instruction into machine-level instructions.

Symbol Table

A Symbol Table is an important data structure used by the compiler to store information about identifiers used in a program.

Identifiers Includes:

- Variables
- Functions
- Arrays
- Objects

The symbol table stores details such as name, type, scope & memory location of identifiers.

Purpose of Symbol Table -

The symbol table helps the compiler to:

- keep track of identifiers used in the program.
- check whether a variable is declared before use.
- Perform type checking.
- Manage memory allocation.

Information Stored in Symbol Table -

The symbol table usually contains the following fields:-

<u>Field</u>	<u>Description</u>
Identifier Name	Name of variable or function
Data type	Type of variable (int, float)
Scope	local or global
memory location	Address of identifier
Value	Value assigned (if any)

Example-

Program;

```
int a;  
float b;
```

Symbol table:

Identifier	Type	Scope	memory location
a	int	local	1000
b	float	local	1004

Data Structures used for Symbol Table -

The Symbol table can be implemented using:

1. Linear list
2. Binary Search Tree
3. Hash Table (most efficient)

Error Handling

Error Handling is the process used by the compiler to detect, report and recover from errors in a program during the compilation process.

When a programmer writes a program, mistakes may occur. The compiler must identify these errors & provide meaningful messages so that the programmer can correct them.

Purpose of Error Handling-

The main objectives of error handling are:

- Detect errors in the program.
- Display meaningful error messages
- Allow compilation to continue if possible
- Help the programmer locate & fix errors easily.

Types of Errors in Compiler-

Lexical Errors

Syntax Errors

Semantic Errors

INPUT BUFFERING

Input Buffering is a technique used in the Lexical Analyzer of a compiler to read the source program efficiently.

Instead of reading characters one by one from the source file, the compiler reads a block of character into memory (buffer).

This improves the speed of compilation and reduces frequent disk access.

Need for Input Buffering-

If the compiler reads character directly from the file:

- It requires many input/output operations.
- Compilation becomes slow.

Using input buffering:

- Characters are read in large blocks.
- Disk access decreases.
- Compilation becomes faster & efficient.

Buffer-

A buffer is a temporary memory area used to store characters read from the source program.

Example:-

Source Program $\rightarrow$ Buffer $\rightarrow$ Lexical Analyzer.

The Lexical Analyzer reads characters from the buffer instead of directly from the file.

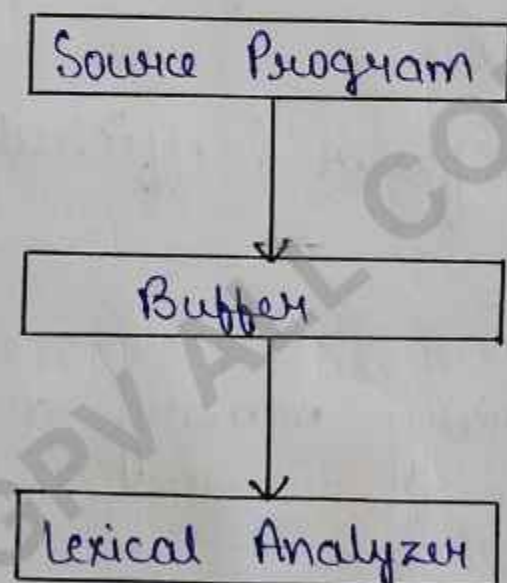
Types of Input Buffering

There are mainly two techniques used in compiler design.

1. Single Buffer Scheme

In this method:

- Only one buffer is used.
- When the buffer becomes empty, the next block of characters is loaded.



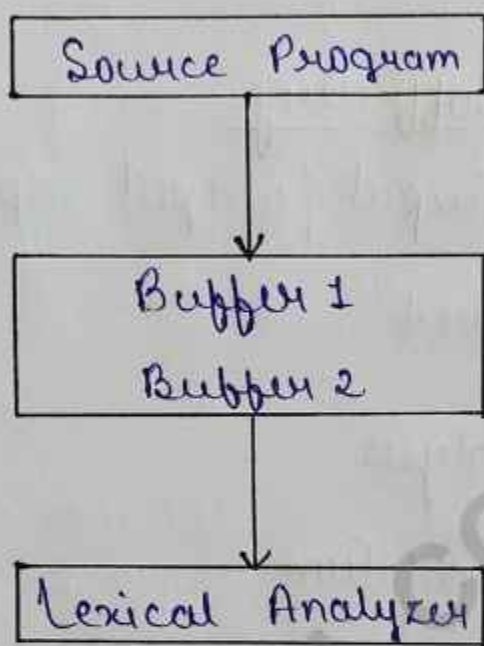
Disadvantages:-

- End-of-buffer checking is required for each character.
- Efficiency is low.

2. Double Buffer Scheme

To overcome the problems of a single buffer, two buffers are used.

Each buffer holds a block of characters.



When one buffer ^{becomes} ~~is~~ empty, the other buffer is filled automatically.

This technique improves efficiency & speed.

Sentinel Technique

In the double buffer scheme, a special character called sentinel (EOF) is placed at the end of each buffer.

This helps the compiler to detect the end of the buffer without checking every character.

This ~~help~~ helps the compiler to detect the end of the buffer without checking every character.

Example:

Buffer 1: $a = b + c$ EOF

Buffer 2: $d * e$; EOF

Advantages of Input Buffering.

- Reduces number of input/output operations
- Improves compiler speed
- Efficient lexical analysis
- Faster Program compilation.

SPECIFICATION & RECOGNITION OF TOKENS

In compiler design, the lexical analyzer reads the source program & converts it into tokens.

To identify tokens correctly, the compiler must:

1. Specify tokens using formal rules.
2. Recognize tokens from the input program.

These two processes are called Specification of tokens & Recognition of tokens.

Token -

A token is the smallest meaningful unit of a program that the compiler can understand.

1. Keyword
2. Identifier
3. Constant
4. Operators
5. Special Symbols

Lexeme -

A lexeme is the actual sequence of characters in the source program that matches a token pattern.

In simple words, a lexeme is the exact text written in the program that represents a token.

The Lexical Analyzer reads the source code & groups characters into lexemes, then converts them into tokens.

Example:- `int count = 10;`

Lexeme
int
count
=
10
;

Token
keyword
identifier
operator
constant
separator

Here,

- count is a lexeme for the token identifier
- 10 is a lexeme for the token constant.

Pattern-

A Pattern is a rule that describes the form of a token in a programming language.

Patterns are usually written using regular expressions, which define how a token should look.

Example:-

total = 25

Token	Pattern	Lexeme
Identifier	$\text{letter}(\text{letter} \text{digit})^*$	total
Operator	$+ - / \% =$	=
Constant	digit^+	25

Key Points:-

- A pattern describes how a token is formed.
- It is usually defined using regular expression.
- It helps the Lexical Analyzer recognize tokens.
- Many lexemes can match the same pattern.

Specification of Tokens

Specification of Tokens is the process of defining tokens of a programming language using formal rules or patterns.

These rules are usually written using regular Expression.

Token specification helps the lexical analyzer identify different tokens in the source program.

Methods for Specification of Tokens

Tokens are specified using regular expressions.

A regular expression is a pattern used to describe a set of strings.

Example of Token Specification -

1. Identifier Pattern -

$\text{letter} (\text{letter} | \text{digit})^*$

Ex:-

a

Sum

Total1

Value23

Rules:-

- Must start with a letter
- Can contain letter & digit.

2. Number

Pattern

digit +

Ex:- 10
25
100

Rule:-

- one or more digits

3. Keywords

Keywords are reserved words of a programming language.

Ex:-

int
float
if
while
return

These words can not be used as identifier.

4. operators

operators perform operations on variable or values.

Ex:-

+

-

=

/

%

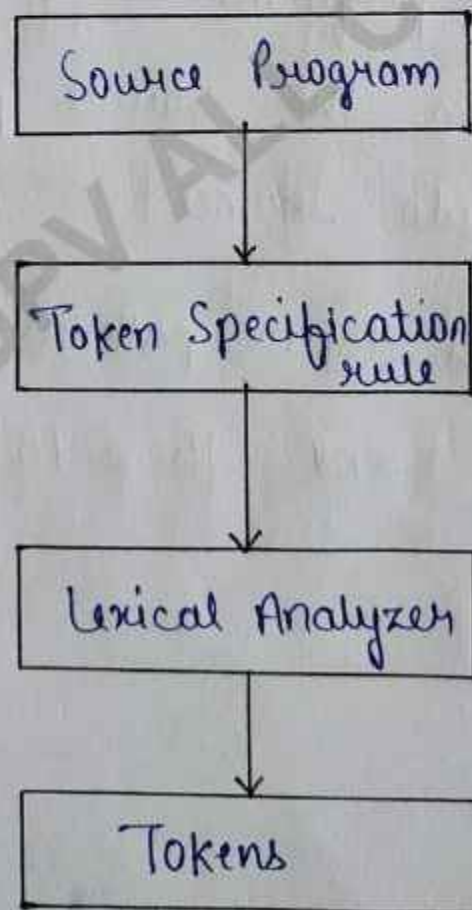
5. Special Symbols

Ex:- ;
,
()
{ }

Role in Compiler -

Token specifications helps the lexical analyzer to:

- Identify tokens correctly
- detect lexical errors
- Convert source program into token stream.



Recognition of Tokens

Recognition of tokens is the process of identifying tokens from the source program based on the patterns defined during token specification.

This process is performed by the lexical analyzer, which reads characters from the source code and groups them into meaningful tokens.

Working of Token Recognition

The lexical analyzer performs the following steps:

1. Read characters from the source program.
2. Group characters into lexemes.
3. Match lexemes with token patterns.
4. Generate tokens & send them to syntax analyzer.

Techniques used for Token Recognition.

Token recognition is usually implemented using:

1) Finite Automata (FA)

Finite Automata are used to recognize patterns of tokens efficiently.

2) Lex tool

A tool that automatically generates a lexical analyzer from regular expressions.

Advantages-

- Identifiers tokens correctly
- Simplifies lexical analysis
- Helps detect lexical errors
- Improves compiler efficiency

Example:-

Pattern for identifier

$\text{letter}(\text{letter}|\text{digit})^*$

Lexemes recognized:-

count

sum 1

value 2

LEX TOOL

LEX is a tool used to automatically generate a lexical analyzer (scanner).

It takes regular expression as input and produces a C program that recognizes tokens in a source program.

LEX is commonly used with YACC (Yet Another Compiler Compiler) to build compilers.

Purpose of LEX-

The main purpose of LEX is to:

- Generate lexical analyzer automatically
- Recognize tokens using regular expressions
- Simplify the process of compiler design.

Structure of LEX Program-

A LEX program is divided into three sections

Definition Section

% %

Rules Section

% %

User Subroutines

1. Definition Section -

This section contains:

- Variable declarations
- Regular definitions
- Header files.

Ex:-

digit [0-9]

letter [a-zA-Z]

These definitions describe patterns used later in the rules section.

2. Rules Section -

This section contains patterns & action.

pattern

action

[0-9]+

printf("Number\n");

[a-zA-Z]+

printf("Identifier\n");

3. User Subroutine Section -

This section contains additional C functions required in the program.

```
int main()
```

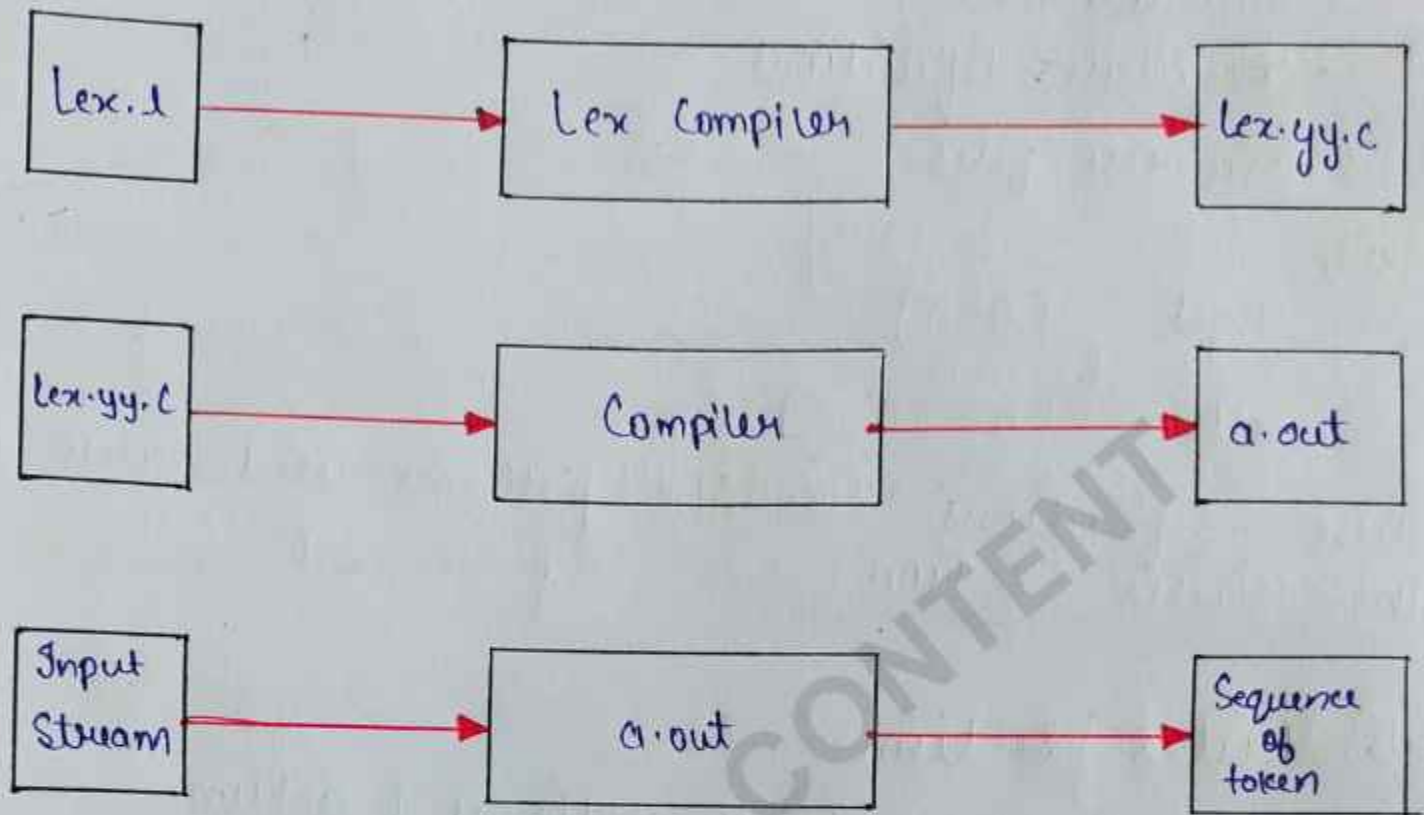
```
{  
    yylex();
```

```
    return 0;
```

```
}
```

Here **yylex()** is the function generated by LEX that performs lexical analysis.

LEX Diagram-



Working of LEX Tool-

Steps involved:

1. Write a LEX program with token rules.
2. LEX converts it into a C program.
3. Compile the generated C program.
4. Run the program to perform lexical analysis.